Source-pinned engineering review

Asymptotic

Beyond the existing review backlog

Native Wolfram interoperability, Mathics contracts, proof boundaries, performance, and validation

Repository `VladimirReshetnikov/Asymptotic`

Reviewed commit

`7d1bc832895cc90a9b2a978b7b7684acab908bd2`

Review date September 9, 2026, America/Los_Angeles

Abstract

This review examines the current implementation in the context of its 27 existing review packages. It does not reproduce their general backlog. Instead, it identifies concrete Mathics adapter costs, a scale-sensitive local-domain proof gap, two test-infrastructure weaknesses, and a bounded special-function extension. It supplies an exact rational sign-certificate construction, terminating hypergeometric coefficient code, an independent frontier-cost model, a fail-closed CI inventory checker, candidate Wolfram Language helpers, and focused reproduction probes.

The distinction between source analysis and execution is essential. Twenty reference-check methods and nine inventory-check methods passed locally; these test the supplied models and prototypes, not the repository package. No package calculation was executed in Mathics or the official kernel during this review. One limited official-kernel primitive probe succeeded. Public witnesses that remain unexecuted are explicitly labeled. No new package-level wrong asymptotic formula is claimed to have been reproduced.

Reading guide Sections 2–4 give the overall assessment; Sections 5–11 contain the incremental findings; Sections 12–15 give integration, acceptance, and development recommendations.

Contents

1 Scope, evidence, and non-duplication

1.1 A fixed source, not a moving target

The review is pinned to commit `7d1bc832895cc90a9b2a978b7b7684acab908bd2`. Repository references in the bibliography use this commit rather than `main`. The source was obtained through the connected GitHub reader. The review concentrated on the public dispatch and result contracts, Mathics compatibility modules, selected inverse and refinement consumers, and the portable validation workflow. This is a risk-based source audit, not a claim that every line of every module or vendored article was verified. [1, 2, 3]

The repository has a substantial history of rapidly incorporated reviews. A defect observed against an earlier snapshot is not automatically a defect of the reviewed snapshot. Conversely, a focused acceptance receipt establishes only the tested behavior and source state. It is not a proof of every parameter regime or every runtime configuration. [5, 6]

1.2 What was compared with the earlier reviews

The non-duplication baseline is the directory index, the consolidated `CODE_REVIEW_STATUS.md` register, and the wave-3 intake/crosswalk. These identify 27 prior review packages. Their finding registers and crosswalks were used to distinguish already-recorded obligations from the new mechanisms below. All 27 original articles were *not* read in full; therefore "new" below means an incremental issue or sharper construction not found in those consolidated registers, not an assertion of absolute novelty across every sentence of all earlier prose. [4, 5, 6]

For example, the register already covers native-tail contracts, precision propagation, exponent equality, composition scope, numerical conditioning, resource accounting, and broader special-function development. Repeating "improve error bounds," "add CI," or "support more transseries" would not satisfy this task. Each finding therefore names its closest existing item and states the additional mechanism, witness, or implementation detail.

The following findings are organized by their immediate cause. N01–N06 are concrete limitations or engineering defects. N07 is a narrowly defined coverage extension, not a wrong-answer allegation. The rewrite-contract proposal in Section 12 is a preventive engineering proposal, not evidence that a current rewrite failed.

1.3 Evidence labels

S: source-established. A control path, guard, data representation, or algorithm is directly present in the pinned source. A complexity claim can be established this way without timing the interpreter.

R: independently reproduced. The supplied Python code executes an exact mathematical model or a synthetic validation fixture. This verifies that model or fixture, not the package implementation as a whole.

U: unexecuted integration witness. A public or private Wolfram Language call is supplied for a fresh-kernel run. Its expected mathematical answer may be proved, but the package's runtime output was not observed here.

P: proposed implementation. Code is supplied for integration and acceptance testing. The Python prototypes were executed; their Wolfram Language translations were not.

1.4 Execution receipt and limits

The local environment used Python 3.13.5 and exact standard-library rational arithmetic. The executed checks comprise 20 methods in `reference_checks.py` and nine in `test_ci_inventory.py`, with zero failures and zero errors. Several methods include multiple deterministic subcases, including 250 generated rational-polynomial inputs and a grid of hypergeometric coefficient checks. The raw outputs are included in `results/`.

The official Wolfram evaluator responded to one primitive query and reported version 15.0.0 for Linux x86 (64-bit), May 6, 2026. Other attempts failed with network/service errors. The successful query did not establish a semantic counterexample for `FirstPosition`; Section 8 explicitly avoids claiming one. The package itself was not loaded or executed in either kernel. No local Mathics installation was available, and a local repository checkout could not be obtained. Accordingly, the supplied CI checker was tested on fixtures, not run against the complete repository.

These limits reduce confidence in public integration predictions, not in simple source facts such as a dense allocation or a nonfinite value passing a Python inequality guard. They also explain why the delivery contains focused probes rather than an invented package pass/fail report.

2 Overall assessment and feature positioning

2.1 The package's strongest contribution

The strongest design contribution is a *reusable generalized-series object with retained mathematical context*, together with specialized inverse engines. A finite expression alone does not retain a coordinate, approach side, error scale, coefficient model, or refinement recipe. `GeneralizedSeries` is intended to carry these distinctions through subsequent operations. The source separates ordinary power-logarithmic jets, logarithmic/Lambert inverse cores, Fourier coefficients, flat sectors, Gamma/Barnes inverse scales, and native delegation. [1, 9, 11, 10, 2]

That is a more defensible positioning than "a replacement for Series." The native system already has broad expansion capabilities. The custom value lies in selected exact generalized scales, constructive inverse coefficient machinery, retained error information, and explicit separation of formal native results from the package's analytic contracts.

For an ordinary positive local coordinate $u \to 0^+$, the basic object can be understood schematically as

$$f(u) = \sum_{\alpha \in A} u^{\alpha} P_{\alpha}(\log u) + O\Big(u^{\beta}(1 + |\log u|)^{d}\Big), \tag{1}$$

where A is finite and ordered, the exponents are exact, and the coefficient polynomials are retained as complete blocks. An operation on such an object must propagate both the finite part and the meaning of the omitted part. This interpretation is mathematical background for the review, not a claim that every source accepted by a native backend automatically satisfies (1).

2.2 What is already implemented and should be preserved

The inspected implementation contains sparse weight handling, complete-layer inverse enumeration, grouped Lagrange and Newton machinery, retained refinement state, and explicit resource failures. The review register also records focused repairs for real coefficients, assumption retention, exponent equality, and composition parameter scope. These are not missing features to propose again. [7, 8, 5]

The Mathics work is similarly more mature than a load-only shim. It uses a package-owned compatibility context, explicit handling of held callables, exact assumption rules, targeted evaluator workarounds, and fresh-kernel portable tests. The documented goal is full compatibility, but the current status is explicitly incomplete. Treating every unresolved proof as a successful compatibility case would contradict that goal and weaken the mathematical contract. [3, 14, 16, 17]

2.3 Principal recommendation

Preserve the separation between native formal output and package analytic output. For the next incremental development cycle, prioritize small compatibility mechanisms that have disproportionately broad reach: local condition proofs, association/frontier costs, sparse coefficient extraction, and test discovery. Broader new scale families should follow these fixes, not obscure them behind more successful examples.

This recommendation does not imply that the existing algorithms are mathematically unsound. The new findings here are chiefly availability, complexity, and validation issues; they matter because they control whether the existing mathematical machinery can be reached and tested reliably.

3 Comparison with built-in Wolfram Language capabilities

3.1 Avoiding an unfair baseline

`Series` is not limited to ordinary Taylor polynomials. Its documented scope includes negative and fractional powers, logarithms, expansions at infinity, special functions, and selected essential-singularity behavior. It supports formal analytic treatment of unknown functions and sequential multivariable specifications. Those facts rule out several common but misleading claims of package novelty. [29]

`InverseSeries` already performs native series reversion, including ramified examples. A quadratic critical point or a fractional inverse series is therefore not, by itself, evidence of a capability unavailable natively. The package comparison should use exact exponent structures, logarithmic coefficient algebras, branch information, and subsequent operations rather than only the presence of a fractional power. [30]

`Asymptotic` supplies more general asymptotic approximations and can operate on structured mathematical problems beyond explicit elementary expressions. Its approximation contract and term-goal semantics must be read in their own terms; they are not the same as a reusable `PowerLogRemainder` object. [31]

`DiscreteAsymptotic` addresses large integer arguments and discrete objects, including sums, products, coefficient extraction and recurrence solutions. A real continuous limit and an integer-sequence limit are not interchangeable contracts. [32]

3.2 Comparison matrix

Task	Native baseline	Reviewed package and proper interpretation
Local algebraic series	`Series`, `SeriesData`, composition and reversion are established native tools.	Custom sparse power-log blocks and retained context can be useful, especially when exponents or coefficient scales do not fit a convenient native lattice.
Implicit/inverse expansions	`InverseSeries` handles ordinary and ramified native series; native symbolic solving may expose an exact inverse.	Direct/grouped Lagrange and Newton inverse engines retain a model, coordinate and coefficient/refinement information. Compare the entire workflow, not only the first few coefficients.
Logarithmic and special inverse scales	Native asymptotic and special-function tools form a substantial baseline.	Dedicated Lambert, Gamma inverse constructions preserve selected exact cores and complete correction blocks. This is specialized machinery, not a general inversion solver for every function.
Oscillatory and flat corrections	Native results may involve oscillatory or exponentially small expressions, depending on the problem.	Fourier-polynomial coefficient algebra and explicit flat-sector operations offer structured manipulation under the admitted scale assumptions. They are not unrestricted transseries closure.
Discrete asymptotics	`DiscreteAsymptotic` has an explicitly discrete target domain.	The inspected native-dispatch switch names only `"Series"` and `"Asymptotic"`. A discrete backend is not present there. This is an existing broad coverage obligation, not a newly discovered mathematical bug.
Certified numerical inversion	Native high-precision numerical solving and interval tools are available, but a numerical answer alone is not a proof certificate.	The package has its own interval certificate and diagnostic interfaces. Their analytic/source scope matters more than the number of displayed digits.
Portable execution	The official kernel is the reference implementation for its own built-ins.	Mathics needs explicit adapter contracts and runtime-specific evidence. A native delegation result on Mathics is limited by what that interpreter actually implements.

The native side of this matrix is supported by the official references; the package side is supported by the pinned implementation and its maintained contract documents. [29, 30, 31, 32, 1, 2, 3, 5]

3.3 A useful order-convention comparison

For a basic analytic example, a native request

```
Series[Exp[x], {x, 0, 3}]
```

retains the cubic coefficient, whereas the package's ordinary exponent-cutoff convention is exclusive: cutoff 3 retains exponents strictly below 3. Thus comparing both calls with the same integer is not necessarily comparing the same achieved precision. This convention is already discussed in the existing API backlog; the useful action here is to make every differential benchmark normalize the target contract first. [29, 1, 5]

A fair comparison record should include the source chart, parameter domain, finite expression, omitted scale, and whether the result is formal or analytic. Equality of `Normal` expressions is not enough. Conversely, a different internal representation is not a failure if the required mathematical information is preserved.

3.4 The native wrapper should remain a wrapper

The inspected native dispatch keeps requests held, distinguishes native results from analytic package results, and records backend decisions. This is an important direction to retain. General coverage of successful native cases should not be achieved by silently reinterpreting every native formal order as a theorem of the form (1). [2]

The previous reviews already identify native dispatch, option identity, eager normal forms, conditional/formal output, and discrete coverage as work items. They are not recounted as new findings here. The additional Mathics-specific recommendation is narrower: determine whether the requested backend is *implemented in this interpreter* before presenting its result as a useful delegated computation. Keep an unresolved native call visibly unresolved; do not let the creation of an inert `System` symbol serve as capability evidence. [6, 3, 12]

4 Mathics compatibility: an evidence-based assessment

4.1 What the architecture gets right

The compatibility modules are loaded only for Mathics. They provide package-owned alternatives while leaving the official Wolfram path and `System` definitions separate. After loading, the compatibility context is removed from the public context path. Late adapters target particular private definitions where evaluator behavior differs. The supplied source and tests explicitly address reloads and namespace isolation. [1, 12, 3, 26]

The exact-assumption adapter is particularly important. It does not replace an unresolved symbolic sign by a floating-point guess. It guards real ordered operands and retains failure for unproved branches. Likewise, the affine interval helper distinguishes truth throughout a deleted interval from failure to prove that truth. Those are desirable properties to preserve when extending the proof domain in Section 5. [14, 15, 17]

Other adapters handle specific evaluator differences: named-function parameter extraction, finite derivative sums, refinement-association construction, empty-list mapping, arithmetic upvalues, and selected special-function Taylor/Stirling models. These are not evidence of universal compatibility, but they show a deliberate compatibility layer rather than an accidental successful load. [16, 21, 23, 22, 24, 18, 19]

4.2 What the existing validation actually establishes

The compatibility document names Mathics3 10.0.1, scanner 10.0.1, SymPy 1.14.0 and Python 3.11 as the development/test configuration. Mathics3 10.0.1 is a real released version, not an inferred version

label. [3, 35]

The repository reports a historical Wolfram preservation run with 1,452 passes and the same 12 existing failures across 79 suites, matching its specified baseline. It separately reports later native definition comparisons and a pending consolidated Mathics acceptance run. These are repository receipts, not tests performed for this article. The source explicitly avoids relabeling the earlier full run as a full run of later changes. [3]

The current workflow already tests both modular and standalone entry points in separate fresh-kernel cases. It has five explicit shards, process timeouts, source-change detection in the runner, and retained diagnostics. "Add a Mathics workflow" would therefore be a stale recommendation. Section 9 instead addresses whether that workflow continues to cover every declared case as the inventory evolves. [28, 27]

4.3 Compatibility is not a single Boolean

Dimension	Evidence at the reviewed snapshot	Remaining acceptance obligation
Load/evaluator semantics	Targeted adapters and loading/reload/context tests exist.	Preserve semantics after source refactors, not only after unchanged reloads.
Exact ordinary algebra	Selected inverse, forward, arithmetic and refinement examples are documented.	Broaden input/option coverage without changing exact coefficients, branch conditions or error meaning.
Proof availability	Bounded realness, sign, polynomial and affine-domain rules exist.	Scale-independent local condition proofs and explicit unresolved states.
Special functions	Selected defining Taylor models, Gamma/Barnes paths and exact-core workarounds exist.	Parameter-domain coverage, terminating cases, and honest distinction between native and package providers.
Numerical behavior	Selected numerical/certificate smoke checks are documented.	Feature-specific accuracy checks; symbolic success must not imply that every exact-core numerical evaluator works.
Performance	Process and term limits exist; implementation uses interpreter-specific workarounds.	Adapter costs must not erase sparse representations or introduce hidden frontier scans.
Display and distribution	Standalone and modular paths are maintained; display fallbacks are documented.	Usable notebook rendering and traceable capability behavior across the supported runtime matrix.

This matrix is an assessment of the documented evidence and inspected design, not a newly executed compatibility certification. [3]

5 N01: local condition proofs depend on an arbitrary radius ladder

Evidence and classification. S: a concrete limitation in the Mathics fast path. R: an exact counterexample to the sufficiency of its radius ladder and a constructive replacement. U: the public package witness remains unexecuted. Classification: availability/coverage, not an unsound accepted expansion.

Source locator: `MathicsInverseBranches.wl`, `inverseFunctionEventually`; `InverseFunctionExpressions.wl`, `forwardPublic` and its original `inverseFunctionEventually`. [17, 25]

5.1 The mechanism and a public witness

After simplifying under $u > 0$, the Mathics wrapper tries the seven radii

$$1,\ 2^{-1},\ 2^{-2},\ \ldots,\ 2^{-6}. \tag{2}$$

A successful affine interval proof returns `True`; otherwise the wrapper falls back to the original quantified proof. That original path asks `Resolve` to establish an existential positive radius. This is a reasonable fallback in the official system, but it is not a replacement for a missing quantified proof capability in Mathics.

Consider

$$0 < u < \varepsilon, \qquad \varepsilon = 2^{-10}. \tag{3}$$

The condition certainly holds on a sufficiently small deleted neighborhood of zero. However, no interval $0 < u < r$ with r from (2) is contained in (3). The affine interval helper therefore cannot prove the condition using any of those trials. This is an exact obstruction, not a numerical tolerance problem.

The public entry point checks eventual conditions before choosing a forward engine, so this concerns even a polynomial source:

```
AsymptoticExpansion[
  ConditionalExpression[x + x^2, 0 < x < 1/1024],
  {x, 0, 3}, "Backend" -> "Package"]
```

The expected finite expression is $x + x^2$, with the input condition retained. No inverse-function difficulty or special-function continuation is involved. The source path indicates a likely `IncompatibleTargetCondition` refusal when the quantified fallback remains unresolved. That exact runtime outcome has not been observed here and is not presented as an executed test.

5.2 Why a longer fixed ladder is not the right fix

Changing the final radius to 2^{-20} merely moves the boundary. For any fixed positive minimum trial radius, a smaller rational ε defeats the same strategy. An affine change of scale can turn a previously accepted local condition into an unproved one without changing the essential germ. The proof should produce a radius from the coefficients rather than search a fixed list unrelated to the input.

This does not require general quantifier elimination. A small exact polynomial rule is enough for a useful initial extension.

5.3 A constructive rational-polynomial certificate

Proposition 5.1 (Deleted-neighborhood sign certificate)**.** *Let $p \in \mathbb{Q}[u]$ be nonzero. Write*

$$p(u) = u^m \left(a_m + \sum_{j=m+1}^{d} a_j u^{j-m} \right), \qquad a_m \neq 0,$$

and set $A = \sum_{j=m+1}^{d} |a_j|$. If $A = 0$, choose $r = 1$; otherwise choose

$$r = \min\left(1, \frac{|a_m|}{2A}\right). \tag{4}$$

Then $\operatorname{sign} p(u) = \operatorname{sign} a_m$ for every $0 < u < r$.

Proof. For $0 < u < r \le 1$, every u^{j-m} with $j > m$ is at most u. Hence

$$\left| \sum_{j=m+1}^{d} a_j u^{j-m} \right| \le Au < Ar \le |a_m|/2$$

when $A > 0$. The bracket has the sign of a_m, and $u^m > 0$. When $A = 0$, the bracket is the nonzero constant a_m, so the conclusion is immediate. □

The certificate records the coefficient list, valuation m, leading coefficient, A, radius, and sign. A checker reconstructs these quantities and verifies $0 < r \le 1$ and $Ar \le |a_m|/2$. It need not trust the radius-producing algorithm. The supplied checker also rejects tampered signs, leading coefficients, tail sums and unsafe radii.

For $p(u) = 2^{-10} - u$, the producer returns $r = 2^{-11}$. This proves the upper inequality in (3); the lower inequality $u > 0$ is immediate. Taking the minimum of the finitely many radii handles a conjunction. Equality and inequality predicates for the identically zero polynomial are treated separately, not assigned an artificial strict sign.

5.4 Safe integration and limitations

The initial implementation should admit exact rational polynomials only, with explicit degree and coefficient-size budgets. A later symbolic-parameter extension can require proved coefficient signs and magnitude bounds, but that is a separate contract. Failure to meet the admitted domain must fall back or remain unresolved; it must not become a false inequality.

Most importantly, this proves *eventual local truth*, not global convexity, global monotonicity, or uniqueness of an inverse branch on an entire domain. It belongs at the local condition boundary. Installing it as a substitute for `inverseBranchGlobalMonotonicity` would be a category error.

Acceptance should include the seven-ladder boundary, much smaller rational neighborhoods, translated endpoints, the opposite approach side, polynomial conditions with higher valuation, false conditions, the zero polynomial, and malformed/inexact inputs. Tests must inspect retained conditions as well as `Normal`. The bundle provides exact Python production/checking code and an isolated candidate WL translation.

Delta from prior reviews. The existing register already discusses broader proof contracts and parameter cases. The additional contribution is the precise seven-radius mechanism, a simple polynomial public witness, and a proved constructive radius replacing the scale-sensitive search. It is not a repetition of "improve branch inference."

6 N02: association compatibility adds a frontier-size factor

Evidence and classification. S: full-key scans in the adapter and a concrete inverse-refinement consumer. R: exact operation counts for that consumer's layer algorithm. U: no Mathics timing or public timeout was measured. Classification: algorithmic performance regression in the compatibility layer.

Source locator: `MathicsCompatibility.wl`, `KeyExistsQ`/`Lookup`; `IncrementalInverse.wl`, `advanceInverseState`. [12, 7]

6.1 The extra work

The Mathics membership adapter computes an `Or` over a mapped `SameQ` comparison with *every* key. Thus a query against a boundary association with B keys performs B equality checks and builds the mapped list, even when the queried key is present early. A surrounding `Or` does not undo work already performed by `Map`.

The inverse frontier algorithm calls this adapter for each candidate neighbor before inserting that neighbor. It also maintains the positive-gap complete-layer invariant. The issue is not that this invariant is wrong; it is that the compatibility membership implementation makes the invariant more expensive to maintain than the surrounding algorithm suggests.

6.2 An exact two-gap workload

For gaps $\{1, 2\}$ and a strict weight cutoff H, the processed region is

$$\mathcal{I}_H = \{(i, j) \in \mathbb{N}^2 : i + 2j < H\}. \tag{5}$$

For $H = 2n$, this region has $n(n+1)$ points. Each point has two candidate neighbors, while the boundary contains a number of keys proportional to H. Consequently the membership-query count is $\Theta(H^2)$, but the full-scan comparison count is $\Theta(H^3)$.

The supplied model reproduces the source's layer removal and neighbor insertion order. It compares the processed region with an independent direct enumeration of (5). It counts only the membership scans, not symbolic coefficients, sorting, string-key construction, cache maintenance, or interpreter overhead.

H	Interior indices	Membership queries	Full-scan comparisons	Peak boundary
16	72	144	1,096	17
32	272	544	8,464	33
64	1,056	2,112	66,592	65
128	4,160	8,320	528,448	129

These are exact structural counts from `reference_results.json`, not seconds measured on Mathics. They establish an avoidable extra factor in this membership component. They do not establish the asymptotic complexity of the complete inverse calculation, whose coefficient arithmetic can be more expensive.

A public workload for subsequent profiling is refinement of the inverse of $x + x^2 + x^3$ on its positive local branch, with `Method->"Lagrange"`. Its ordinary normalized gaps are 1 and 2. Match the internal weight cutoff to the public observable convention rather than assuming they are the same integer.

6.3 Repairs and tradeoffs

A simple short-circuit membership loop removes unnecessary comparisons on early hits, but does not remove the worst-case linear scan or repeated `Keys` allocation. It is a modest improvement, not a complete complexity repair. Replacing the predicate without understanding Mathics association semantics could also break keys containing lists, held expressions or missing values.

A better consumer-level repair is to collect all candidate neighbors of the current complete weight layer, deduplicate their exact index keys once, and merge them with the remaining frontier in one batch.

Because every generator gap is strictly positive, no new neighbor belongs to the weight layer just removed. Because predecessor weights are smaller, an admitted new neighbor cannot already be an interior point. These facts justify batching without changing the layer semantics.

The chosen data structure must be tested in Mathics, rather than assuming that an association offers the same operation cost as a hash table in another runtime. A sorted exact-key merge is an alternative when interpreter-native key operations are weak. Preserve deterministic layer order and the existing resource-failure contract. A bulk merge must not silently admit an oversized frontier and discover that fact only after allocating it.

Delta from prior reviews. P04 already identifies avoidable direct-region work in grouped and Newton methods; P06 addresses general budget units. This finding is a specific Mathics-added membership factor inside the retained Lagrange frontier, supported by a call path and an independent exact count. It does not re-propose the existing incremental state as though it were missing.

7 N03: sparse coefficient rules are reconstructed through a dense list

Evidence and classification. S: definite dense intermediate allocation in the adapter. R: exact support-versus-slot counts. U: end-to-end Mathics memory/time behavior was not measured. Classification: representation/performance limitation.

Source locator: `MathicsAlgebra.wl`, one-variable `CoefficientRules`; `GammaInverse.wl`, `gammaInverseBound`. [13, 10]

The compatibility implementation of `CoefficientRules[p,q]` first constructs `CoefficientList[Expand[p],q]`, attaches exponents to every position, then discards zero coefficients and reverses the list. The *returned* object is sparse, but the path to it is not.

For

$$p_D(q) = 1 + q^D, \tag{6}$$

the natural sparse support has two entries, while the coefficient list has $D + 1$ entries. At $D = 10^6$, a two-term input therefore requests 1,000,001 coefficient slots before subsequent mapped rules and filtering. No measured byte count is claimed: expression representation and sharing are interpreter-dependent. The number of slots is not.

This is easy to miss in low-order regression tests because the resulting rules are mathematically correct. It becomes especially relevant when a consumer needs only the smallest occupied exponent. The inspected `gammaInverseBound` uses coefficient rules to obtain that minimum; it does not intrinsically need a dense array or every zero position.

There is an important qualification. Other package paths, including coefficient normalization in the Fourier module, also use dense `CoefficientList` operations. Therefore it would be incorrect to claim that removing this one adapter makes all polynomial handling sparse, or that every large-degree public failure originates in this adapter. [11]

7.1 A bounded sparse replacement

For an already expanded, admitted univariate polynomial, visit its nonzero monomials. For each monomial, determine the nonnegative integer exponent, extract the variable-free coefficient, group equal exponents, and sort only the occupied exponents in the required order. For S occupied powers this uses $O(S)$ storage for the coefficient records, excluding coefficient expression size and the cost of expansion.

The qualification "already expanded" matters. Expanding $(1+q)^D$ has $D+1$ nonzero terms and is genuinely dense; no representation trick can promise a two-term result. A sparse reader must therefore preflight or budget expansion separately, and refuse unsupported forms rather than misread a rational function, generalized power, or coefficient depending on q.

For a valuation-only consumer, use a smaller query: compute the least occupied exponent after exact cancellation, without constructing a complete coefficient-rule representation. Its zero-polynomial case must be explicit, because `Min[{}]` is not a useful frontier degree. Such a query should not become an unbounded simplification oracle.

7.2 Acceptance cases

Use $1+q^D$ over a sequence of degrees and verify that the adapter does not construct $D+1$ intermediate positions. Include zero, a constant, cancellation of equal powers, coefficients containing independent parameters, a genuinely dense polynomial, a multivariate request, a rational function, and invalid variables. Differential tests must verify ordering and zero handling as well as value equality. The existing output-support budget does not by itself bound a dense temporary representation.

Delta from prior reviews. The earlier register already flags dense `SeriesData` export and generic resource accounting. This is a different ingress to dense allocation: the Mathics replacement for `CoefficientRules`, including a consumer that only requires a valuation. It is not a repetition of the native export-span issue.

8 N04: first-position lookup materializes every match

Evidence and classification. S: all-match materialization in a first-match adapter. U: package performance impact is not timed. The limited official-kernel probe did not establish a callback-semantic difference. Classification: avoidable work, not a reproduced semantic bug.

Source locator: `MathicsCompatibility.wl`, `FirstPosition`; `IncrementalInverse.wl`, `incrementalResizePolynomialCache`. [12, 7]

The adapter implements a first-position query by obtaining the complete `Position` result and then selecting its first entry. For a list of N matching elements, this builds N position records even though the answer is the first position. The inspected cache-resizing consumer repeatedly looks up the first maximum in a list of polynomial-cache lengths; tied lengths create exactly the kind of multiple-match result that is discarded.

The official `Position` interface provides a maximum-number-of-positions argument. A candidate implementation is therefore structurally of the form

```
positions = Position[expr, pattern, levels, 1, Heads -> heads];
If[positions === {}, default, First[positions]]
```

provided that the admitted bounded form works correctly in the supported Mathics version. The source's existing lazy-default behavior must be retained; a normal eager function wrapper can accidentally evaluate `default` even when a match is found. [33, 34]

This replacement is a *proposal requiring a Mathics primitive test*, not an assumption that every native `Position` form is implemented there. The tests should cover root-level matches, explicit levels, heads, no match, a delayed default with a counter, and associations with keyed positions. A custom depth-first

traversal is a fallback only if the bounded primitive is unavailable, and must preserve the admitted traversal contract.

There is no need to exaggerate this issue into a semantic counterexample. The one successful official-kernel experiment returned the same first position for both compared expressions. No side-effect discrepancy was observed. The static storage distinction is sufficient to justify a repair. The probe and its actual returned output are included in `results/native_probe.txt`.

Delta from prior reviews. The existing performance register contains broad normalization and caching concerns. This is a specific adapter whose output requirement is strictly smaller than its intermediate result, together with a real cache-maintenance consumer. It is separate from N02's association membership scan and from native series densification.

9 N05: green CI can omit newly declared portable tests

Evidence and classification. S: discovery and sharding mechanisms permit omissions. R: synthetic mutations reproduce the omissions and the proposed checker rejects them. No current repository test or group is claimed to be omitted. Classification: latent validation correctness defect.

Source locator: `validation/run_mathics_tests.py`, `CASE_PATTERN` and `available_cases`; `.github/workflows/mathics.yml`, suite matrix and bash group assignments. [27, 28]

9.1 Formatting-sensitive discovery

The runner discovers test IDs and groups with the literal regular expression

```
^portableTest\["([a-z0-9-]+)", "([a-z]+)",
```

using multiline matching. This requires the head at the start of a line, the first two arguments on that line, and particular spacing after the first comma. A valid Wolfram declaration may not match:

```
 portableTest["indented", "forward", actual, expected];
portableTest[
  "multiline", "forward", actual, expected];
portableTest["compact","forward",actual,expected];
```

If at least one other declaration matches, the existing nonempty-inventory check still succeeds. The unseen case is not selected or launched, so the runner's otherwise strict result protocol cannot detect its absence.

The independent fixture contains a conventional control declaration plus these three formatting variants. The transcribed current regular expression finds only the control. This is a demonstrated property of the discovery expression, not a claim that these exact formatting variants are present in the pinned suite.

9.2 The fixed shard map is not checked against the full inventory

The workflow assigns groups to five shards:

Shard	Groups
foundation	loading, primitive, assumptions
calculus	forward, inverse, arithmetic
contracts	callable, contracts, native, flat, certificate, numerical, logarithmic
special	special, families
operations	operations

Suppose a correctly formatted new test is assigned group `recurrence`. It is a valid newly discovered group, but no workflow shard requests it. The runner checks whether *requested* groups are known; it does not require that the union of requested groups covers the entire inventory. Every old shard can remain nonempty and pass while the new group is never run.

This is stronger than the generic warning that zero tests might count as success. The omitted case coexists with many genuinely executed, successful tests. Counts and freshness checks cannot reveal it unless the expected inventory is independently related to the shard selection.

9.3 A fail-closed inventory contract

The most maintainable long-term arrangement is one manifest driving both test registration/discovery and CI partitioning. At minimum, require

$$\forall t \in \mathcal{T}, \qquad \#\{s \in \mathcal{S} : t \text{ is assigned to } s\} = 1 \tag{7}$$

for each package entry-point variant. The inventory must itself be complete under an explicit registration grammar; applying (7) to an incomplete regular-expression result does not solve the first problem.

The supplied `ci_inventory.py` is a small fail-closed checker for the current literal layout. It tokenizes strings, whitespace and nested Wolfram comments; accepts direct literal `portableTest` declarations; excludes the function's `SetDelayed` definition; and rejects unsupported dynamic registrations rather than silently omitting them. It reads the literal suite matrix and bash shard definitions from the workflow, checks that they agree, and enforces exactly-once coverage. It also rejects duplicate IDs, duplicate assignments and empty shards.

This is not a general Wolfram Language parser or a general YAML interpreter. Its admitted grammar is intentionally small. A future dynamic registration or workflow refactor should either adopt a manifest or explicitly update the checker. Failing loudly is preferable to silently reducing coverage.

The nine checker tests include whitespace/comments, strings that resemble tests, dynamic-registration refusal, duplicate IDs, workflow-matrix mismatch, unassigned groups, and a valid partition. They passed locally. The checker has not been run on a local copy of the complete repository, which was not available; the integration command is included in the README.

9.4 Acceptance should include mutation tests

Add an indented declaration, then a multiline declaration, then a new group, and verify that each is either executed or rejected by an inventory check. Move a group into two shards and verify a failure. Comment out a declaration and verify that the inventory shrinks intentionally rather than trying to launch an impossible case. These mutations test the meaning of the workflow, not merely that a fixed existing test file still passes.

Delta from prior reviews. V01 already asks for native semantic CI and V02 addresses empty or bad test receipts. The current Mathics workflow exists and its runner rejects empty selections. The additional failure mode is a nonempty, passing workflow that silently omits legal declarations or newly introduced groups.

10 N06: the process timeout accepts nonfinite floating-point values

Evidence and classification. S: a concrete Python validation gap. R: the guard behavior and the proposed validator were tested. No hanging kernel process was launched. Classification: low-cost robustness fix.

Source locator: `validation/run_mathics_tests.py`, argument parsing and the `args.timeout<=0` guard. [27]

The runner parses `--timeout` as a Python `float` and rejects only values less than or equal to zero. Both

```
float("nan") <= 0
float("inf") <= 0
```

are false. Thus nonfinite values pass a guard intended to establish a positive process deadline.

The resulting `communicate(timeout=...)` behavior depends on the subprocess implementation and process state. It may fail in timer conversion or cease to provide a meaningful finite deadline. The review does not claim a particular platform hang or exception was reproduced. The logical defect is simply that the accepted input does not meet the runner's finite-timeout contract.

The minimal repair is

```
import math
...
if not math.isfinite(args.timeout) or args.timeout <= 0:
    parser.error("--timeout must be finite and positive")
```

A proposed two-hunk patch is supplied. The reusable Python argument type in `reference_checks.py` rejects NaN, positive and negative infinity, zero, negatives and malformed input, and accepts positive fractional values.

Finite validation is not a complete policy for arbitrarily enormous values. A production runner may additionally impose a documented maximum supported duration based on the operating-system timer interface. That is a separate range policy; it should not silently change ordinary explicit user deadlines.

Delta from prior reviews. This is a precise input-validation hole in the newer portable runner, not the already-recorded generic request for timeouts or reliable exit codes. Existing process-tree termination and strict output checking should be retained.

11 N07: terminating hypergeometric polynomials are outside the Taylor gate

Evidence and classification. S: explicit admission restrictions in the adapter. R: exact terminating coefficient calculations and rejection tests. U: the public Mathics expansion witnesses have not been executed. Classification: bounded coverage extension, not an incorrect coefficient formula.

Source locator: `MathicsTaylor.wl`, `mathicsDefiningTaylor` and `mathicsTaylorSeries`. [18]

11.1 The current gate is sufficient but unnecessarily restrictive

The helper admits a constant multiple of a supported function evaluated at a vanishing monomial. For generalized hypergeometric functions it requires, among other conditions, $p \le q + 1$ and strictly positive lower parameters. It also imposes a finite order cap of 400. These are conservative sufficient conditions for the intended nonterminating defining-series path; they are not a complete characterization of finite polynomial cases.

The standard defining coefficients are

$$c_k = \frac{\prod_{i=1}^{p} (a_i)_k}{k! \prod_{j=1}^{q} (b_j)_k}. \tag{8}$$

With lower parameters away from nonpositive integers, a nonpositive integer upper parameter causes termination, including when $p > q + 1$. [36]

For example,

$${}_3F_0(-2, 1, 3; \, ; z) = 1 - 6z + 24z^2. \tag{9}$$

The length test $3 > 0 + 1$ excludes this parameter set from the custom defining Taylor path. A fallback may still succeed if the runtime simplifies the function to a polynomial first. That possibility is why the finding is stated at the provider boundary and the public failure is not claimed as observed.

Similarly,

$${}_2F_1(-2, 1; -\tfrac{1}{2}; z) = 1 + 4z - 8z^2 \tag{10}$$

has no lower-parameter pole but fails the helper's lower-parameter positivity test. Both identities are verified by the supplied exact recurrence and independent direct products.

11.2 A finite provider with a clear domain

An initial extension can be deliberately small: admit exact rational parameters, require at least one nonpositive integer upper parameter, and refuse every nonpositive integer lower parameter. Let

$$N = \min\{-a_i : a_i \in \{0, -1, -2, \ldots\}\}.$$

Compute $c_0 = 1$ and, for $0 \le k < N$,

$$c_{k+1} = c_k \frac{\prod_i (a_i + k)}{(k + 1) \prod_j (b_j + k)}. \tag{11}$$

The denominators are nonzero under the admitted lower-parameter rule. The numerator contains a terminating factor at the next step after the last retained coefficient. Thus the result is an exact polynomial, not an infinite series with an empirically inferred tail.

Classify termination *before* applying the nonterminating p, q convergence test. The finite branch does not need that test. Likewise, a request for a very high order need not allocate a high-order coefficient array when the function has already been proved to be a degree-two polynomial. The relevant resource bound is the terminating degree and coefficient growth, not the requested Taylor order alone.

11.3 Do not guess a convention at lower-parameter poles

It is tempting to admit a nonpositive integer lower parameter whenever termination appears to occur before a zero denominator. That shortcut can silently select a parameter-limit convention. Cancellation of equal upper and lower parameters, specialization of singular parameters, and taking a limit need not be interchangeable operations.

The proposed implementation therefore refuses *all* nonpositive integer lower parameters in its first version. This is intentionally conservative, including cases for which a convention could later be defined. A separate regularized or explicitly parameter-limited interface can be added only with its own contract. Negative noninteger rationals, by contrast, do not make a lower Pochhammer factor vanish and are admitted.

11.4 Integration must preserve exactness

The existing helper always returns a `SeriesData` order term. The terminating provider should instead expose the fact that the polynomial is exact. Merely fabricating a distant native order endpoint would discard useful exactness information, while blindly returning a polynomial to a consumer that expects `SeriesData` could break dispatch.

A suitable integration is to recognize the terminating atom before the nonterminating Taylor provider and feed the exact polynomial into the ordinary finite-expression parser. If a provider interface is introduced, its return value should explicitly distinguish "exact finite polynomial" from "finite expansion with tail." Preserve the original expression for provenance and reject surrounding transformations that invalidate the admitted rewrite conditions.

The Python implementation checks recurrence coefficients against direct Pochhammer products for multiple degrees and positive/negative noninteger lower parameters. It tests the first vanishing coefficient and rejects parameter poles, nonterminating inputs and degree-budget violations. The WL helper is an unexecuted translation in a separate context; it does not modify the repository or `System` definitions.

Delta from prior reviews. The existing development backlog includes wider special-function support. This contribution is an exact finite subdomain missed by a specific new Mathics admission gate, with two explicit identities, a coefficient algorithm, pole-convention safeguards, and an exactness-aware integration plan. It is not a request for arbitrary hypergeometric continuation.

12 Late adapter rewrites need installation contracts

12.1 The maintenance risk

Several Mathics adapters replace selected symbols or exact expression shapes inside `DownValues`. For example, the refinement adapter targets `Association[rules_Map]` in one consumer and `Associatio n[rules_Part]` in another. The empty-list mapping workaround scans package-private downvalues, while arithmetic support is installed through a separate upvalue path. [23, 22, 24]

This is a practical way to keep official-kernel definitions separate. It also introduces a coupling between source syntax and compatibility behavior. A mathematically harmless refactor can change an AST shape so that a rewrite matches zero sites. Without an expected-match check, installation can still complete normally. Conversely, a broader replacement can affect an extra consumer after a source change.

No such missed installation is claimed to have been reproduced at the pinned commit. The recommendation is to make these assumptions executable before a refactor silently invalidates them. It is distinct from the

earlier standalone `Get` scanner issue: this concerns semantic adapter installation inside an already loaded package.

12.2 A concrete rewrite contract

For each targeted transformation, retain an installation record with *consumer*, *precondition pattern*, *expected match count or allowed range*, *replacement*, *postcondition*, and *primitive regression*. Require the precondition count before mutation and verify the postcondition after mutation. A zero-match result should be intentional and recorded, or an installation failure, not silently treated as success.

Use held structural inspection so that auditing definitions does not execute user-owned values. The existing code already takes care around held installation and named formal parameters; a registry must preserve that care rather than introducing string-evaluation shortcuts. [22, 16]

The most useful mutation tests change a producer's syntax while preserving its native mathematical result: wrap a mapped rule list in a local variable, move an association constructor, or change a helper call's spelling. The adapter test should either continue to prove the desired postcondition or fail with a specific installation diagnostic.

12.3 Capability sentinels, not version strings alone

A runtime name is a routing hint, not a semantic proof. In a validation-only suite, test the exact primitive facts an adapter relies on: lazy lookup fallback, held extraction, bounded position search, empty-list mapping, association updates, and selected numeric core identities. Maintain the expected outcome by supported runtime version. If a Mathics release repairs a primitive, that should prompt a controlled adapter retirement decision, not an automatic assumption that all adapters are now unnecessary.

This is a more specific implementation of the already-recorded generic capability-metadata idea. It need not become a large public registry or run expensive probes during every package load. A small developer-facing installation report and CI sentinels are sufficient to start.

13 API and UX: improvements tied to the new findings

13.1 An unproved condition is not an incompatible condition

The public eventual-condition check currently uses a Boolean result. When a proof backend cannot establish a true condition, the resulting failure message can read as though the condition itself were incompatible with the requested approach. N01 shows why those outcomes need to be distinguished. [25]

A proposed diagnostic should distinguish `ProvedTrue`, `ProvedFalse`, and `Unresolved`, with the last attempted proof method and relevant source condition. This does not require accepting more inputs or changing successful mathematical results. It avoids telling the user that a simple valid local domain is mathematically wrong when the actual limitation is an unavailable proof capability.

The prior register already requests richer failure metadata in general. The additional UX recommendation is this specific unresolved-versus-false boundary, with a counterexample and a small proof method that resolves it.

13.2 Resource messages should identify the representation that grew

For N02–N04, `MaxTerms` and an elapsed-time failure alone can be misleading. A user may have asked for a small sparse output while the adapter created a dense coefficient list or traversed an entire boundary for every membership test. A developer diagnostic should report the phase, input support, relevant degree or boundary size, and the representation being allocated.

This is not a proposal to change the existing global budget semantics under the user's feet. Start with diagnostics and phase counters. Only then decide whether an additional budget is needed. Avoid one counter named "terms" that mixes occupied monomials, dense slots, index pairs, expression leaves, and proof attempts without identifying which one triggered the failure.

13.3 Keep exact formulas and numerical evaluators separate

The Mathics core adapter normalizes selected principal-branch `ProductLog[0,z]` constructions to the one-argument spelling, while leaving nonprincipal branches to the interpreter. The repository documents a numeric limitation in Mathics's two-argument path. This is already a known compatibility limitation, so it is not a new finding here. [20, 3]

The useful distinction is at the result surface. A Lambert finite logarithmic approximation need not contain `ProductLog`, even though its `ExactInverseExpression`, `CoreInverseExpression`, or seed metadata does. It would therefore be wrong to infer that every `s[value]` on such a result is numerically broken. Equally, a symbolic identity in metadata is not evidence that the current interpreter numerically evaluates it correctly. [9]

For validation, test each numerical reference route separately from the returned finite approximation. An unreliable exact-core evaluator must not serve as the sole oracle that certifies the approximation. No new checked numerical API is claimed to be invented here; that general request is already in the existing backlog. This is a concrete runtime-dependent oracle separation requirement.

13.4 Preserve deliberate limitations

A structured refusal for an unproved branch, nonreal coefficient, unsupported scale transformation, or ambiguous parameter limit is preferable to a plausible-looking formula with an unjustified error claim. The purpose of N01 and N07 is to enlarge *proved* subdomains, not to remove conservative checks indiscriminately. Likewise, a native delegation wrapper should keep its native status visibly distinct rather than borrowing the package's stronger analytic vocabulary.

14 Acceptance, performance measurement, and prioritization

14.1 A minimum merge gate for the proposed work

The following requirements are concrete acceptance criteria, not a claim that they all passed during this review.

Change	Required positive and negative evidence	Guard against regression
N01 local proof	Exact small-radius and scaled-domain cases; zero/nonzero polynomial relations; false predicates; unsupported coefficients remain unresolved.	Preserve source/target conditions and branch-selection scope. No global monotonicity inference from a local sign certificate.
N02 membership	Identical complete index layers, coefficients, and resource-failure behavior; counters over increasing cutoffs.	No dropped resonant indices, duplicate keys, changed cancellation order, or uncontrolled bulk frontier allocation.
N03 sparse rules	Values/order agree on sparse and dense polynomials; memory/counters remain support-sensitive on $1 + q^D$.	Preserve coefficient domain, zero polynomial handling, and multivariate fallback.
N04 first position	Bounded primitive works on supported levels/heads; no-match defaults stay lazy.	Preserve position representation and held evaluation; do not assert a speedup before measuring it.
N05 inventory	Whitespace/comment and new-group mutations are detected; all cases have exactly one shard per entry point.	Unsupported dynamic registration fails loudly rather than returning a partial inventory.
N06 timeout	NaN/infinities/zero/negative values fail before process launch; finite positive values retain behavior.	Do not disturb process-group termination or explicit valid deadlines.
N07 finite PFQ	Exact coefficient recurrence equals direct products; pole conventions rejected; exactness retained.	No accidental use of a nonterminating convergence theorem or fabricated tail.
Rewrite contracts	Expected sites and postconditions checked before/after refactors.	Official-kernel definitions and public namespace behavior remain unchanged.

Run these in fresh official and Mathics kernels, against both modular and standalone builds. Use the existing portable runner where possible rather than creating an unrelated second infrastructure. For development-only private primitives, record that scope explicitly; passing a private helper does not replace the corresponding public-path test. [28, 27]

14.2 What to measure for performance

A useful benchmark separates process startup, package loading, source normalization, coefficient work, and optional output conversion. Fresh-kernel latency is a real user cost, but it should not be mistaken for the asymptotic cost of one adapter. Conversely, a warm-kernel microbenchmark does not show that a notebook workflow is responsive.

For N02, record interior indices, peak frontier, membership queries and membership comparisons. For N03, record occupied powers, maximum degree, intermediate coefficient slots and peak memory. For N04, record inspected nodes and returned/all candidate position counts. These counters relate a measured slowdown to a source mechanism rather than just to a function name.

Do not generalize a Python operation-count model into a claimed Mathics speed ratio. The reference model deliberately excludes coefficient expression growth, sorting, key conversion and interpreter overhead. A repair can eliminate an asymptotic component without dominating wall time at the tested orders. That is still useful, but it must be reported accurately.

14.3 Priority order

First: protect the evidence and eliminate trivial robustness gaps. Integrate the inventory/-partition check and finite-timeout guard. These are small changes with independent tests and limited mathematical risk. Add rewrite-installation postconditions for the highest-impact adapters.

Second: remove artificial compatibility barriers. Integrate the exact rational-polynomial local proof path, with three-way proof diagnostics. Optimize the association/frontier membership mechanism and the sparse coefficient-rules adapter. Validate the first-position bounded primitive before using it.

Third: enlarge a proved mathematical subdomain. Add the terminating rational-parameter hypergeometric provider, preserving exactness and refusing singular lower-parameter conventions. Extend only after the exact finite path has native/Mathics parity and clear resource bounds.

These stages are not elapsed-time estimates. They order dependencies and risk. They also avoid re-listing the already-recorded long-term program of integration, complex sectors, multivariate implicit problems, uniform parameters, general transseries, and discrete adapters. Those remain important existing obligations, not new recommendations of this article. [5, 6]

15 Novelty ledger and conclusions

ID	Incremental finding	Closest prior topic	Additional contribution
N01	Fixed-radius local proof coverage	C13, D02, X07	Seven-radii obstruction, polynomial public witness, exact certificate producer/checker.
N02	Full association scans in refinement	P04, P06, P08	Mathics-specific extra factor in an existing retained frontier; exact operation counts and batch-repair invariant.
N03	Dense route to sparse coefficient rules	C01, P06	Different dense intermediate in `CoefficientRules`, plus valuation-only consumer and bounded sparse design.
N04	All-match work for a first match	P08	Specific compatibility helper and cache consumer; bounded-position acceptance contract, without a false semantic claim.
N05	Incomplete discovery/shard coverage	V01, V02	Nonempty passing suites can omit legal declarations/new groups; lexical and partition mutation checks.
N06	Nonfinite timeout acceptance	V02, P06	Exact NaN/infinity guard bypass and minimal validation patch.
N07	Terminating PFQ subdomain	Existing coverage roadmap	Explicit finite cases excluded by a convergence gate, exact recurrence, singular-parameter safeguards and exactness-aware integration.
A01	Rewrite installation contracts	D02, W3-11 (adjacent)	Consumer-level rewrite pre/postconditions and syntax-preserving mutation tests; not another standalone-load scanner finding.

The references in the third column are proximity markers, not assertions that the earlier items are identical to these findings. The full older backlog is intentionally not reproduced. [5, 6]

15.1 Bottom line

The repository contains substantial specialized mathematics and unusually explicit attention to remainder and branch contracts. The most productive incremental review is therefore not another generic list of missing CAS features. It is a review of the small mechanisms that decide whether that mathematics remains reachable, portable, and honestly tested.

The source-established new concerns are chiefly hidden adapter costs and validation omissions. The strongest mathematical extension is a small exact local-sign certificate that removes an arbitrary radius scale from condition checking. The terminating hypergeometric extension is similarly useful because it is finite, exact, and has a sharply stated parameter domain.

No new package-level incorrect asymptotic formula was reproduced in this review. That statement is a scope limit, not a claim that none exists. The supplied public witnesses, candidate helpers and acceptance matrix give a concrete route to determine the remaining runtime facts without weakening the package's conservative proof boundaries.

A Source coverage and reproduction map

The following table identifies the principal source areas examined. Some large files were read in selected ranges rather than in full. Bibliographic links point to the fixed commit and function locators in the findings identify the relevant definitions.

Area	Principal files/functions
Entry, contracts and native dispatch	`AsymptoticAnalysis.wl` (selected public, native-import and load-order ranges); `NativeCompatibility.wl` (dispatch and status ranges).
Mathics primitives and algebra	`MathicsCompatibility.wl`; `MathicsAlgebra.wl`; `MathicsAssumptions.wl`; `MathicsSimplification.wl`; `MathicsCalls.wl`.
Mathics late adapters	`MathicsInverseBranches.wl`; `MathicsTaylor.wl`; `MathicsCoreFunctions.wl`; `MathicsSpecialFunctions.wl`; `MathicsCalculus.wl`; `MathicsLists.wl`; `MathicsFormatting.wl`; `MathicsRefinement.wl`.
Inverse/refinement consumers	`IncrementalInverse.wl`; selected `RefinementState.wl`, `LambertInverse.wl`, `GammaInverse.wl`, `FourierCoefficients.wl` ranges.
Callable and condition paths	`InverseFunctionSyntax.wl`; selected `InverseFunctionBranches.wl` and `InverseFunctionExpressions.wl` ranges.
Validation and prior work	`run_mathics_tests.py`; selected `MathicsTests.wl`; `mathics.yml`; Mathics compatibility/callables documents; review index, consolidated status and wave-3 intake.

A.1 Executed artifacts

`code/reference_checks.py` is standard-library Python and can be run without the repository or any CAS. Its JSON output explicitly records `package_executed:false` and `mathics_executed:false`. The frontier results are exact structural counts. The sign certificates are checked by a sufficient rational inequality, not numerical samples.

`code/ci_inventory.py` and `code/test_ci_inventory.py` supply the lexical/partition checker and its nine fixture tests. The checker deliberately rejects unsupported registration/workflow layouts. This is a proposed integration aid, not proof that a complete repository inventory was loaded and checked here.

A.2 Unexecuted artifacts

`code/ReviewProbes.wl` contains fresh-kernel characterization probes. Set `ASYMPTOTIC_REVIEW_SOURCE` to an absolute package entry path and `ASYMPTOTIC_REVIEW_CASE` to one of its documented case IDs. It prints the observed result, not a fabricated pass/fail claim. Place an external process timeout around cost experiments.

`code/ProposedMathicsHelpers.wl` is an isolated WL translation of the rational-germ and finite-PFQ constructions. It does not modify `System` or package definitions. Its Python counterparts passed the supplied tests; that is not a substitute for executing the translation on both target kernels.

`patches/finite-timeout.patch` is a minimal proposed runner change. Check and apply it against the pinned checkout before using it; subsequent source changes may require context adjustment.

B Suggested focused public probes

These calls are useful acceptance targets, not recorded package outputs. Use fresh source/target symbols and a fresh kernel for each case.

B.1 Local condition covariance

```
AsymptoticExpansion[
  ConditionalExpression[x + x^2, 0 < x < 2^-k],
  {x, 0, 3}, "Backend" -> "Package"]
(* k = 1, 5, 6, 7, 10, 30; all define the same polynomial germ. *)
```

Check the finite expression and exactness, but also verify that each result retains its own condition. Conditions that are false on the selected side must not be admitted by the new proof path.

B.2 Terminating polynomial provider

```
AsymptoticExpansion[
  HypergeometricPFQ[{-2, 1, 3}, {}, x],
  {x, 0, 3}, "Backend" -> "Package"]
(* Mathematical polynomial: 1 - 6 x + 24 x^2. *)
```

Test the lower-noninteger example separately. Test the private provider in Mathics to distinguish a new finite-provider success from an unrelated native pre-evaluation to the same polynomial.

B.3 Frontier and sparse-adapter costs

```
s = AsymptoticInverse[x + x^2 + x^3, {x, 0}, {y, 8},
  Method -> "Lagrange"];
```

```
SeriesRefine[s, 16]
```

Instrument frontier membership rather than interpreting wall time alone. For sparse coefficients, characterize the package-owned Mathics `CoefficientRules` helper on $1 + q^D$, increasing D under a process memory/time guard. Do not substitute an already dense polynomial and call its unavoidable output size a sparse-regression witness.

C Artifact and editorial notes

The new reference Python code, checker, candidate WL helpers, and textual patch in this bundle are supplied as review artifacts, not as an upstream release. Their test scope is documented per file. No repository source tree, third-party binary, font file, or checksum file is bundled.

The article uses standard mathematical proofs and original explanatory text. Brief code fragments identify mechanisms under review; the full source remains available at the pinned repository links. Existing review findings are attributed by register identifier rather than republished as new discoveries.

References

[1] Vladimir Reshetnikov / Asymptotic contributors. Asymptotic repository. `src/Kernel/AsymptoticAnalysis.wl`. Reviewed commit `7d1bc83`. https://github.com/VladimirReshetnikov/Asymptotic/blob/7d1bc832895cc90a9b2a978b7b7684acab908bd2/src/Kernel/AsymptoticAnalysis.wl

[2] Vladimir Reshetnikov / Asymptotic contributors. Asymptotic repository. `src/Kernel/NativeCompatibility.wl`. Reviewed commit `7d1bc83`. https://github.com/VladimirReshetnikov/Asymptotic/blob/7d1bc832895cc90a9b2a978b7b7684acab908bd2/src/Kernel/NativeCompatibility.wl

[3] Vladimir Reshetnikov / Asymptotic contributors. Mathics compatibility status and evidence. `docs/Mathics/COMPATIBILITY.md`. Reviewed commit `7d1bc83`. https://github.com/VladimirReshetnikov/Asymptotic/blob/7d1bc832895cc90a9b2a978b7b7684acab908bd2/docs/Mathics/COMPATIBILITY.md

[4] Vladimir Reshetnikov / Asymptotic contributors. Prior review package index. `external-reports/code-review/README.md`. Reviewed commit `7d1bc83`. https://github.com/VladimirReshetnikov/Asymptotic/blob/7d1bc832895cc90a9b2a978b7b7684acab908bd2/external-reports/code-review/README.md

[5] Vladimir Reshetnikov / Asymptotic contributors. Consolidated code-review status register. `docs/development/CODE_REVIEW_STATUS.md`. Reviewed commit `7d1bc83`. https://github.com/VladimirReshetnikov/Asymptotic/blob/7d1bc832895cc90a9b2a978b7b7684acab908bd2/docs/development/CODE_REVIEW_STATUS.md

[6] Vladimir Reshetnikov / Asymptotic contributors. Wave-3 intake and finding crosswalk. `docs/development/WAVE_3_INTAKE.md`. Reviewed commit `7d1bc83`. https://github.com/VladimirReshetnikov/Asymptotic/blob/7d1bc832895cc90a9b2a978b7b7684acab908bd2/docs/development/WAVE_3_INTAKE.md

[7] Vladimir Reshetnikov / Asymptotic contributors. Incremental inverse state and coefficient algorithms. `src/Kernel/IncrementalInverse.wl`. Reviewed commit `7d1bc83`. https://github.com/VladimirReshetnikov/Asymptotic/blob/7d1bc832895cc90a9b2a978b7b7684acab908bd2/src/Kernel/IncrementalInverse.wl

[8] Vladimir Reshetnikov / Asymptotic contributors. Retained inverse refinement state. `src/Kernel/RefinementState.wl`. Reviewed commit `7d1bc83`. https://github.com/VladimirReshetnikov/Asymptotic/blob/7d1bc832895cc90a9b2a978b7b7684acab908bd2/src/Kernel/RefinementState.wl

[9] Vladimir Reshetnikov / Asymptotic contributors. Lambert inverse construction and metadata. src/Kernel/LambertInverse.wl. Reviewed commit 7d1bc83. https://github.com/VladimirReshetnikov/Asymptotic/blob/7d1bc832895cc90a9b2a978b7b7684acab908bd2/src/Kernel/LambertInverse.wl
[10] Vladimir Reshetnikov / Asymptotic contributors. Gamma inverse construction. src/Kernel/GammaInverse.wl. Reviewed commit 7d1bc83. https://github.com/VladimirReshetnikov/Asymptotic/blob/7d1bc832895cc90a9b2a978b7b7684acab908bd2/src/Kernel/GammaInverse.wl
[11] Vladimir Reshetnikov / Asymptotic contributors. Fourier-polynomial coefficient algebra. src/Kernel/FourierCoefficients.wl. Reviewed commit 7d1bc83. https://github.com/VladimirReshetnikov/Asymptotic/blob/7d1bc832895cc90a9b2a978b7b7684acab908bd2/src/Kernel/FourierCoefficients.wl
[12] Vladimir Reshetnikov / Asymptotic contributors. Mathics primitive compatibility definitions. src/Kernel/MathicsCompatibility.wl. Reviewed commit 7d1bc83. https://github.com/VladimirReshetnikov/Asymptotic/blob/7d1bc832895cc90a9b2a978b7b7684acab908bd2/src/Kernel/MathicsCompatibility.wl
[13] Vladimir Reshetnikov / Asymptotic contributors. Mathics algebra adapters. src/Kernel/MathicsAlgebra.wl. Reviewed commit 7d1bc83. https://github.com/VladimirReshetnikov/Asymptotic/blob/7d1bc832895cc90a9b2a978b7b7684acab908bd2/src/Kernel/MathicsAlgebra.wl
[14] Vladimir Reshetnikov / Asymptotic contributors. Mathics exact assumption calculus. src/Kernel/MathicsAssumptions.wl. Reviewed commit 7d1bc83. https://github.com/VladimirReshetnikov/Asymptotic/blob/7d1bc832895cc90a9b2a978b7b7684acab908bd2/src/Kernel/MathicsAssumptions.wl
[15] Vladimir Reshetnikov / Asymptotic contributors. Mathics simplification adapters. src/Kernel/MathicsSimplification.wl. Reviewed commit 7d1bc83. https://github.com/VladimirReshetnikov/Asymptotic/blob/7d1bc832895cc90a9b2a978b7b7684acab908bd2/src/Kernel/MathicsSimplification.wl
[16] Vladimir Reshetnikov / Asymptotic contributors. Mathics held callable extraction. src/Kernel/MathicsCalls.wl. Reviewed commit 7d1bc83. https://github.com/VladimirReshetnikov/Asymptotic/blob/7d1bc832895cc90a9b2a978b7b7684acab908bd2/src/Kernel/MathicsCalls.wl
[17] Vladimir Reshetnikov / Asymptotic contributors. Mathics real inverse-branch helpers. src/Kernel/MathicsInverseBranches.wl. Reviewed commit 7d1bc83. https://github.com/VladimirReshetnikov/Asymptotic/blob/7d1bc832895cc90a9b2a978b7b7684acab908bd2/src/Kernel/MathicsInverseBranches.wl
[18] Vladimir Reshetnikov / Asymptotic contributors. Mathics defining Taylor providers. src/Kernel/MathicsTaylor.wl. Reviewed commit 7d1bc83. https://github.com/VladimirReshetnikov/Asymptotic/blob/7d1bc832895cc90a9b2a978b7b7684acab908bd2/src/Kernel/MathicsTaylor.wl
[19] Vladimir Reshetnikov / Asymptotic contributors. Mathics special-function adapters. src/Kernel/MathicsSpecialFunctions.wl. Reviewed commit 7d1bc83. https://github.com/VladimirReshetnikov/Asymptotic/blob/7d1bc832895cc90a9b2a978b7b7684acab908bd2/src/Kernel/MathicsSpecialFunctions.wl
[20] Vladimir Reshetnikov / Asymptotic contributors. Mathics exact-core adapters. src/Kernel/MathicsCoreFunctions.wl. Reviewed commit 7d1bc83. https://github.com/VladimirReshetnikov/Asymptotic/blob/7d1bc832895cc90a9b2a978b7b7684acab908bd2/src/Kernel/MathicsCoreFunctions.wl
[21] Vladimir Reshetnikov / Asymptotic contributors. Mathics finite-sum calculus adapters. src/Kernel/MathicsCalculus.wl. Reviewed commit 7d1bc83. https://github.com/VladimirReshetnikov/Asymptotic/blob/7d1bc832895cc90a9b2a978b7b7684acab908bd2/src/Kernel/MathicsCalculus.wl
[22] Vladimir Reshetnikov / Asymptotic contributors. Mathics empty-list mapping workaround. src/Kernel/MathicsLists.wl. Reviewed commit 7d1bc83. https://github.com/VladimirReshetnikov/Asymptotic/blob/7d1bc832895cc90a9b2a978b7b7684acab9

08bd2/src/Kernel/MathicsLists.wl
[23] Vladimir Reshetnikov / Asymptotic contributors. Mathics refinement constructor rewrites. src/Kernel/MathicsRefinement.wl. Reviewed commit 7d1bc83. https://github.com/VladimirReshetnikov/Asymptotic/blob/7d1bc832895cc90a9b2a978b7b7684acab908bd2/src/Kernel/MathicsRefinement.wl
[24] Vladimir Reshetnikov / Asymptotic contributors. Mathics arithmetic upvalues and formatting. src/Kernel/MathicsFormatting.wl. Reviewed commit 7d1bc83. https://github.com/VladimirReshetnikov/Asymptotic/blob/7d1bc832895cc90a9b2a978b7b7684acab908bd2/src/Kernel/MathicsFormatting.wl
[25] Vladimir Reshetnikov / Asymptotic contributors. Inverse-expression and eventual-condition entry points. src/Kernel/InverseFunctionExpressions.wl. Reviewed commit 7d1bc83. https://github.com/VladimirReshetnikov/Asymptotic/blob/7d1bc832895cc90a9b2a978b7b7684acab908bd2/src/Kernel/InverseFunctionExpressions.wl
[26] Vladimir Reshetnikov / Asymptotic contributors. Portable exact regression declarations. validation/MathicsTests.wl. Reviewed commit 7d1bc83. https://github.com/VladimirReshetnikov/Asymptotic/blob/7d1bc832895cc90a9b2a978b7b7684acab908bd2/validation/MathicsTests.wl
[27] Vladimir Reshetnikov / Asymptotic contributors. Fresh-kernel portable regression runner. validation/run_mathics_tests.py. Reviewed commit 7d1bc83. https://github.com/VladimirReshetnikov/Asymptotic/blob/7d1bc832895cc90a9b2a978b7b7684acab908bd2/validation/run_mathics_tests.py
[28] Vladimir Reshetnikov / Asymptotic contributors. Mathics CI workflow. .github/workflows/mathics.yml. Reviewed commit 7d1bc83. https://github.com/VladimirReshetnikov/Asymptotic/blob/7d1bc832895cc90a9b2a978b7b7684acab908bd2/.github/workflows/mathics.yml
[29] Wolfram Research. *Series*. Official online reference consulted for this review. https://reference.wolfram.com/language/ref/Series.html
[30] Wolfram Research. *InverseSeries*. Official online reference consulted for this review. https://reference.wolfram.com/language/ref/InverseSeries.html
[31] Wolfram Research. *Asymptotic*. Official online reference consulted for this review. https://reference.wolfram.com/language/ref/Asymptotic.html
[32] Wolfram Research. *DiscreteAsymptotic*. Official online reference consulted for this review. https://reference.wolfram.com/language/ref/DiscreteAsymptotic.html
[33] Wolfram Research. *Position*. Official online reference consulted for this review. https://reference.wolfram.com/language/ref/Position.html
[34] Wolfram Research. *FirstPosition*. Official online reference consulted for this review. https://reference.wolfram.com/language/ref/FirstPosition.html
[35] Mathics3 project. *Mathics3 10.0.1 release metadata*. Official online reference consulted for this review. https://pypi.org/project/Mathics3/10.0.1/
[36] NIST Digital Library of Mathematical Functions. *Section 16.2: Definition and Analytic Properties*. Official online reference consulted for this review. https://dlmf.nist.gov/16.2